B05 | 创建之后，怎样读回同一条记录？

本章整合 **创建输入 → 服务建立记录 → 回应交出 id → 用这个 id 查询 → 比较读回结果**。重点是把上一次的结果交给下一次，不是再记一套框架名。

怎样确认两次请求操作的是同一条记录？

当前问题	关系或短用法	记到什么程度
查询编号从哪里来？	`ticket_id = created["id"]` 取本次创建编号，再交给 `get_ticket(client, ticket_id)` 发查询。	**上一次实际结果决定下一次目标。** 不把 `T-42` 写死为每次的答案。
读回成功，只看 200 够吗？	本例没有中途修改时，`assert fetched == created` 比较五个公开字段；类型正确还不证明是本次记录。	认熟比较关系，完整检查脚本查用；取字典值本身不发送请求。
创建 201、查询 200、退出 0 各证明什么？	服务日志记录 HTTP 状态；脚本比较公开结果。退出 0 不等于严格断言了每个具体状态。	**证据检查到哪，结论才到哪。** 响应成功、结果相同、程序正常结束分别看。
服务检查过了，客户端为什么还查？	服务检查自己的处理；客户端检查实际收到的状态、JSON 类型、解析结果和字段，保护后续使用。	不把 `.json()` 当成完整检查；`id/title/status` 要字符串，`priority/version` 要整数且排除 bool。

本例用 `type(value) is int` 排除 `True`：Python 中 bool 与 int 有继承关系，但字段合同不接受布尔优先级。该写法按需查；关键是 **结构类型与业务对应关系是不同的检查**。关闭 HTTPX Client 只收尾客户端资源，不会停止另一个窗口的服务。

回看：「两次交换怎样连成一件事？」「用本次返回的编号，再发一次查询」「为什么服务检查过了，客户端还要检查？」。

要运行时，怎样找到同一份项目和自己的服务？

当前问题	关系或短用法	记到什么程度
在哪里建立环境、加脚本？	从可见入口 01_配套代码_A-E → examples → b_http_api → ticket-api 进入，把整个项目复制到自己的新实验目录；有副本则继续原副本，不覆盖或合并已有修改。它与命令里的底层路径 01_配套练习 指向同一份文件。	用自己的副本，不在随书参考目录安装或产生运行状态；路径是定位入口，不是必背内容。
两个窗口怎样确认没走错？	各自用 `Get-Location` 看位置、`Get-ChildItem` 看文件，都进入同一副本；应有 `app.py`、`client.py`、`requirements.txt`。	逐渐认熟查看位置与文件；新窗口不会自动跟随另一窗口后来改变的位置。
安装、测试、启动怎样保持同一个 Python？	建 `.venv` 后，明确用其中的 `python.exe` 执行 pip、pytest、Uvicorn；`pip check` 查依赖兼容，pytest 查已写断言。	认熟“同一解释器运行工具”，完整路径、选项、版本号查用。
`app:app` **怎样读，要开几个窗口？**	前者是模块，后者是应用对象。普通创建查询用服务 + 客户端两窗口；双服务摘要用两服务 + 客户端三窗口。	窗口用于观察不同进程，不是业务按窗口先后自动执行；完整启动命令不用背。
端口占用或结束后仍监听，怎么办？	启动前查监听；结束自己的前台服务用 `Ctrl+C`，提示符恢复后再查；仍有记录则核对 PID 和实际身份。	**停止动作不等于端口已释放。** 不按示例 PID 或端口盲目结束进程，查询与核对方法查 A10。

本例用 `127.0.0.1` 绑定本机，8000 / 8001 匹配两个服务配置，不要求背数字。双服务准备在专用新窗口指定本机供应商，并移除该窗口继承的供应商 token；只影响该窗口环境，不删除秘密文件或改系统设置。需要运行时完整看原章准备，不从半行提示猜操作。

按问题找文件即可：`app.py` 看服务处理，`client.py` 看工单调用，`supplier_client.py` 看供应商检查，`fake_supplier.py` 看受控结果。

回看：「在电脑上，先把项目放到哪里？」「怎样准备同一个 Python 环境？」「为什么要打开两个窗口？」「两个服务的调用，要观察哪两段？」。进程核对另见 A10「从第二个终端追到真正的回应」「发出停止动作以后，还要看一次结果」。

失败后留下什么，下一步靠什么判断？

当前问题	关系或观察入口	记到什么程度
创建输入被拒绝，后面的查询会发生吗？	本例 `"urgent"` 优先级在输入校验处 422，创建正文未运行；客户端状态检查停止，没有这次成功的 `created`。	沿最早断点判断：取编号、查询、比较还没有机会执行。
创建成功，后来查询失败，会撤销创建吗？	查不存在的编号返回 404；前一次创建仍已发生。	**多个请求连起来，不自动成为同进同退的一次操作。**
创建请求超时，可以当作没创建再发吗？	没取得完整回应，服务也可能已经执行；先找服务端证据，不盲目重复创建。	与函数前的输入拒绝分开判断，timeout 不证明业务效果为零。
故意检查 502、504，为何不用成功状态检查？	这次要接收并比较预期失败；`raise_for_status()` 常用于阻止正常业务继续用坏结果。	**按目的选动作**，测试失败路线不必照抄正常成功路线。
测试、真实调用、日志怎样配合？	测试检查既定条件与分支，本机真实调用补进程和连接证据，日志帮助追已记录的阶段；替换位置的具体区别见 B04。	它们不互相替代，没看到日志也不能单独证明零调用。
重启后记录去哪了？	`app.state.tickets` 的内存随服务停止消失，新应用建立空状态。	**HTTP 交接正确不等于持久保存。** C 册用独立练习连接数据库，不假定共享此项目的库或状态。

双服务验证时，每次外部请求用自己的 request_id，在那次跨服务处理中沿用。外层 502 可能对应内层 500；外层 504 不表示供应商返回过 504。输出几个状态数字只是结果，还需把它们对应回实际发生的事件。

回看：「改一个地方，看看失败从哪里开始」「两个服务的调用，要观察哪两段？」「测试、真实调用和日志，各多看到了什么？」「服务重启后，为什么记录不见了？」。

B 册收束的是 **值怎样进入消息、经过对方处理，又成为本地可继续使用的值**。要重新辨认 API / HTTP / JSON、FastAPI / Uvicorn / HTTPX 的配合，回看全书「知识关系总览 | 把消息、函数和证据放回同一张图」。不用把库名当成互不相干的课程重背一次。